

# App Dev 2 Project Report

## 1. Student Details

**Name:** Darshil Srivastava

**Roll Number:** 24F2006384

**Email:** 24f2006384@ds.study.iitm.ac.in

**About Me:** I am a student of the IIT Madras BS Degree program with a keen interest in full-stack web development. I enjoy building practical applications that solve real-world problems, combining clean backend APIs with intuitive user interfaces.

## 2. Project Details

**Project Title:** Placement Portal Application (PPA)

**Problem Statement:**

Institutes require efficient systems to manage campus recruitment activities involving companies and students. Currently, many institutes rely on spreadsheets, emails, or manual coordination, which makes it difficult to manage company approvals, placement drives, student registrations, and application tracking. This application solves that by providing a unified platform for Admin, Companies, and Students to interact with the system based on their roles.

**Approach:**

The application was built using Flask as the backend REST API framework with a modular structure. Vue 3 (via CDN with ES Modules) handles the frontend as a Single Page Application. The backend and frontend are separated into distinct folders. Redis is used for caching and as a Celery broker. Celery handles scheduled and async background jobs including daily reminders, monthly reports, and CSV exports.

## 3. AI/LLM Declaration

I used Claude (Anthropic) to assist in generating boilerplate code for SQLAlchemy model definitions, Vue component templates, and JWT authentication setup.

The extent of AI/LLM usage is around **20–30%**, limited to **code scaffolding and structural suggestions**. All final implementation logic, debugging, integration, and testing were done manually.

## 4. Technologies and Frameworks Used

| Technology / Library | Purpose                                                |
|----------------------|--------------------------------------------------------|
| Flask                | Core backend REST API framework                        |
| Flask-JWT-Extended   | JWT-based authentication and role-based access control |
| SQLAlchemy           | ORM for SQLite database (models and relationships)     |
| SQLite               | Lightweight local database for all application data    |
| Jinja2               | Renders single entry-point HTML template for the SPA   |

| Technology / Library | Purpose                                                         |
|----------------------|-----------------------------------------------------------------|
| Vue 3 (CDN)          | Frontend Single Page Application with component-based UI        |
| Vue Router           | Client-side routing between pages (hash history mode)           |
| Axios                | HTTP client for API calls from Vue to Flask                     |
| Bootstrap 5          | Frontend styling and responsive layout                          |
| Redis                | Caching (with TTL) and Celery message broker/backend            |
| Celery               | Async and scheduled background job processing                   |
| Werkzeug             | Password hashing (generate_password_hash / check_password_hash) |

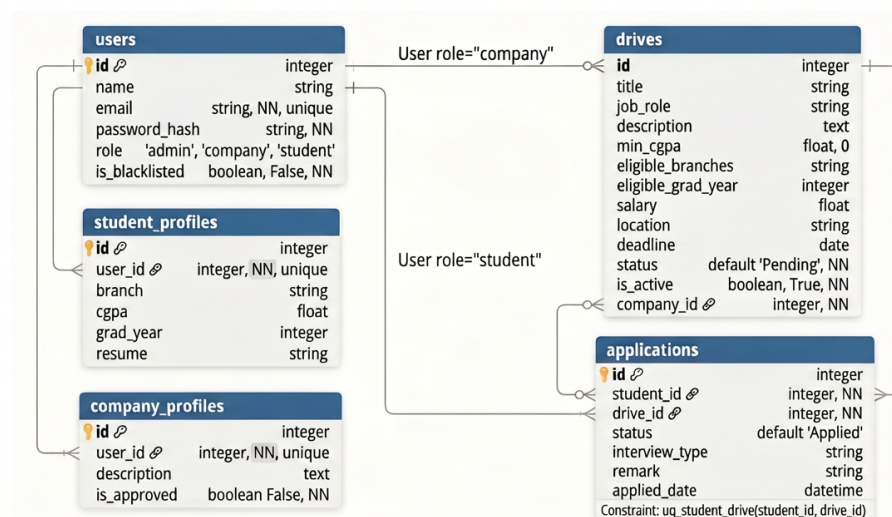
## 5. Database Schema / ER Diagram

Tables:

- User — unified user model for all roles (id, name, email, password\_hash, role, is\_blacklisted)
- StudentProfile — student-specific data (id, user\_id, branch, cgpa, grad\_year, resume)
- CompanyProfile — company-specific data (id, user\_id, description, is\_approved)
- Drive — placement drive created by a company (id, title, job\_role, description, min\_cgpa, eligible\_branches, eligible\_grad\_year, salary, location, deadline, status, is\_active, company\_id)
- Application — student application to a drive (id, student\_id, drive\_id, status, interview\_type, remark, applied\_date)

Relationships:

- One-to-One → User → StudentProfile
- One-to-One → User → CompanyProfile
- One-to-Many → User (Company) → Drive
- One-to-Many → Drive → Application
- One-to-Many → User (Student) → Application



## 6. API Resource Endpoints

### Auth

| Endpoint      | Method | Description                            |
|---------------|--------|----------------------------------------|
| /api/register | POST   | Register a new student or company      |
| /api/login    | POST   | Login and receive JWT access token     |
| /api/me       | GET    | Get current authenticated user details |

### Admin

| Endpoint                             | Method | Description                                      |
|--------------------------------------|--------|--------------------------------------------------|
| /api/admin/dashboard                 | GET    | Dashboard stats, all companies, students, drives |
| /api/admin/companies/<id>/approve    | POST   | Approve a company registration                   |
| /api/admin/companies/<id>/reject     | POST   | Reject a company registration                    |
| /api/admin/companies/<id>/blacklist  | POST   | Blacklist a company                              |
| /api/admin/companies/<id>/reactivate | POST   | Reactivate a blacklisted company                 |
| /api/admin/students/<id>/blacklist   | POST   | Blacklist a student                              |
| /api/admin/students/<id>/reactivate  | POST   | Reactivate a blacklisted student                 |
| /api/admin/drives/<id>/approve       | POST   | Approve a placement drive                        |
| /api/admin/drives/<id>/reject        | POST   | Reject a placement drive                         |
| /api/admin/drives/<id>               | GET    | View a specific drive                            |
| /api/admin/applications/<id>         | GET    | View a specific application                      |

### Company

| Endpoint                              | Method | Description                                         |
|---------------------------------------|--------|-----------------------------------------------------|
| /api/company/dashboard                | GET    | Company dashboard with drives                       |
| /api/company/profile                  | PUT    | Update company profile/description                  |
| /api/company/drives                   | GET    | List all company drives                             |
| /api/company/drives                   | POST   | Create a new placement drive                        |
| /api/company/drives/<id>/close        | POST   | Close/complete a drive                              |
| /api/company/drives/<id>/applications | GET    | View all applicants for a drive                     |
| /api/company/applications/<id>        | GET    | View a specific application                         |
| /api/company/applications/<id>        | PUT    | Update application status (shortlist/select/reject) |

## Student

| Endpoint                       | Method | Description                                  |
|--------------------------------|--------|----------------------------------------------|
| /api/student/dashboard         | GET    | Dashboard with approved drives + eligibility |
| /api/student/profile           | PUT    | Update student profile                       |
| /api/student/resume            | POST   | Upload resume file                           |
| /api/student/history           | GET    | View full application history                |
| /api/student/drives/<id>       | GET    | View drive detail with eligibility check     |
| /api/student/drives/<id>/apply | POST   | Apply to a placement drive                   |
| /api/student/companies/<id>    | GET    | View company profile and drives              |
| /api/student/export            | POST   | Trigger async CSV export of applications     |
| /api/student/export/<task_id>  | GET    | Poll CSV export task status                  |

## 7. Architecture and Features

### Folder Structure:

- backend/app.py — Flask app factory, admin seeding, SPA catch-all route
- backend/celery\_worker.py — Celery app with Beat schedule for background jobs
- backend/Application/model.py — SQLAlchemy models (User, StudentProfile, CompanyProfile, Drive, Application)
- backend/Application/controller.py — All 31 REST API routes
- backend/Application/auth.py — JWT helpers and role\_required decorator
- backend/Application/cache.py — Redis cache wrapper with TTL and graceful degradation
- backend/Application/tasks.py — 3 Celery tasks (reminders, report, CSV export)
- frontend/index.html — Single Jinja2 entry point with importmap for ES modules
- frontend/static/js/main.js — Vue app entry point
- frontend/static/js/store.js — Reactive auth store + Axios instance
- frontend/static/js/router.js — Vue Router with role-based navigation guards
- frontend/static/js/components/ — Shared components (AppAlert, StatusBadge)
- frontend/static/js/views/ — Page components organized by role (admin/, company/, student/)

### Implemented Features:

- Role-based authentication using JWT (Admin, Company, Student)
- Unified User model with role-specific profile tables
- Admin: approve/reject/blacklist companies, drives, and students
- Company: register, create placement drives (pending admin approval), manage applicants
- Student: browse approved drives with eligibility filtering and search, apply, track status
- Drive approval two-stage workflow (company creates → admin approves)
- Duplicate application prevention (DB-level unique constraint)
- Eligibility validation (CGPA, branch, graduation year) before allowing apply
- Redis caching with TTL and cache invalidation on data mutation
- Resume upload (PDF/DOC/DOCX) stored in static/resumes/

- Student profile editing including password change

#### Background Jobs (Celery + Redis):

- Daily reminder job (Celery Beat, 8 AM) — emails students about drives with deadlines in 3 days
- Monthly report job (Celery Beat, 1st of month) — HTML email report to admin with placement stats
- Async CSV export (user-triggered) — generates application history CSV, emails it, returns download link

## 8. Video Presentation

### Drive Link:

[ <https://drive.google.com/file/d/1c6ztQz-hGuxwBv6X-bkgWERt40Gg-6ex/view?usp=sharing> ]